based on this pdf we have to do this task satisfying all the conditions and I should also take photos for the dataset how long will it take do this task and which type of photos should I click and what is the step by step procedure to solve this task

I've read the full PDF carefully. Here's a complete breakdown of the task, time estimate, and step-by-step procedure.

⏱ Realistic Time Estimate

Phase	Estimated Time
Photo capture (80+ images)	1–2 hours
Manual labeling (CSV)	2–3 hours
Dataset organization & split	30 min
Model training (ResNet18)	2–4 hours (code + run)
Optional visualization overlay	1–2 hours
Total	~7–11 hours

This is a 1–2 day task if spread across sessions. Labeling is the most tedious part.

📸 What Photos to Click & How

Best scene type for you (as a robotics student): indoor corridor/hallway navigation scenes. [1] This aligns perfectly with robotics obstacle avoidance use cases and is easy to capture in your campus building.

What to look for in each photo:

- Clear near-to-far depth layers visible — e.g., a hallway stretching away, a table with objects at different distances, or a staircase [1]
- Each of the 4 quadrants (TL, TR, BL, BR) should ideally show a *different* depth zone [1]
- Avoid flat scenes where everything is at the same depth (a blank wall, for example)

Variation to include across your 80+ photos:

- Different lighting (bright corridor, dim hallway, outdoor)
- Different camera angles (straight ahead, slightly tilted down, from a corner)
- Different object arrangements in the scene
- Mix of near obstacles (chair, box, backpack on floor) and far background [1]

Avoid:

- Selfies or portrait shots
- Scenes with no visible depth gradient
- Mixing tabletop AND outdoor randomly — pick one domain and stick to it [1]

Step-by-Step Procedure

Step 1 — Choose Your Scene Domain

Pick **indoor navigation** (recommended for robotics relevance): hallways, stairwells, rooms with furniture arranged at varying depths. [1]

Step 2 — Capture 80+ Images

Use your smartphone. Walk through Campus Cham or your apartment building and photograph from multiple positions. Aim for consistent framing (hold phone at chest/eye level). Target ~80–100 images. [1]

Step 3 — Define Depth Categories Clearly

Write down your exact rule *before* labeling and stick to it. Example for hallways: [1]

- **Near (N):** objects within ~1.5m (floor just in front, chair leg)
- **Middle (M):** 1.5m–4m away
- **Far (F):** walls, doors, corridor end beyond 4m

Step 4 — Label Each Image (CSV Format)

Split each image visually into 4 quadrants (TL, TR, BL, BR) and assign N/M/F to each. Save as CSV: [1]

```
image_001.jpg,F,F,M,N
image_002.jpg,M,M,N,N
```

You can use a simple spreadsheet (Excel/Sheets) and rename files systematically. Labeling 80 images at ~1.5 min each = ~2 hours.

Step 5 — Organize Dataset Split

Split your labeled CSV into 3 sets: [1]

- **Train:** 56 images (70%)
- **Val:** 12 images (15%)
- **Test:** 12 images (15%)

Use `sklearn.model_selection.train_test_split` or manually shuffle and split.

Step 6 — Build the CNN Model (ResNet18 + 4 Heads)

Use PyTorch. The architecture: [1]

```
import torchvision.models as models
import torch.nn as nn

backbone = models.resnet18(pretrained=True)
features = backbone.fc.in_features
backbone.fc = nn.Identity()  # remove default head

# 4 output heads, one per grid region (TL, TR, BL, BR)
# each head outputs 3 classes: Near=0, Middle=1, Far=2
head_TL = nn.Linear(features, 3)
head_TR = nn.Linear(features, 3)
head_BL = nn.Linear(features, 3)
head_BR = nn.Linear(features, 3)
```

- Loss: **CrossEntropyLoss for each head**, summed [1]
- Optimizer: Adam, lr=1e-4
- Input images: resize to 224×224, normalize with ImageNet stats

Step 7 — Train & Evaluate

- Train for 20–50 epochs on your small dataset
- Monitor validation accuracy per head
- Report per-region accuracy on the test set [1]

Step 8 — Optional: Depth Visualization Overlay

Color-code predictions on top of the original image: [1]

- 🔴 Red = Near (danger zone for robotics)
- 🟡 Yellow = Middle (caution)
- 🟢 Green = Far (safe to navigate)

Draw colored semi-transparent rectangles over each of the 4 quadrants using OpenCV or Matplotlib.

⚠ Key Tips (Ruthless Advice)

- **Don't be lazy with labeling.** Inconsistent labels will destroy your model's accuracy. Define your rules FIRST, then label all at once.
- **80 images is very small.** Use data augmentation (random flip, color jitter, random crop) or your model will overfit badly.
- **ResNet18 pretrained is the right call** — training from scratch with 80 images would be garbage.
- The hardest part is NOT the code — it's taking *good, diverse photos* with clear depth structure. Spend time on this.

⁂

i took 54 images and by using augmentation we will increase the the number of images and then we will see how it works and how do i do the labelling and classification

Good. 54 images is honestly on the low side even with augmentation, but it's workable. Here's the complete labeling + classification pipeline tailored to your task.

The Hard Truth First

54 images is below the required 80. Augmentation helps training generalize better, but it does **not** substitute for real image diversity. Your test/validation set must use **original, non-augmented images only** — augmented copies go in training only. If your model fails, low data variety is the first suspect.[2]

Part 1 — Labeling Your 54 Images

How to Label Efficiently

You don't need any fancy tool. Do this manually with a spreadsheet:

Step 1 — Rename your images systematically

```
image_001.jpg, image_002.jpg ... image_054.jpg
```

Step 2 — Open each image and mentally split it into 4 quadrants:

```
| TL | TR |
| BL | BR |
```

Step 3 — For each quadrant, assign one label based on the dominant depth:

- **N (Near):** Closest objects, foreground floor, object right in front
- **M (Middle):** Mid-range area, objects/walls at medium distance
- **F (Far):** Background, far wall, corridor end, sky [2]

Step 4 — Create a CSV file like this:

```
filename,TL,TR,BL,BR
image_001.jpg,F,F,M,N
image_002.jpg,M,M,N,N
image_003.jpg,F,F,F,F
```

Labeling Tips

- Label **all 54 images in one sitting** so your mental rule stays consistent [2]
- If a quadrant is 50/50 ambiguous, pick the **dominant depth zone** — what covers more area
- Typical indoor hallway pattern: top quadrants = Far, bottom quadrants = Near/Middle — this is normal and expected
- Don't second-guess every image. Be fast and consistent

Part 2 — Augmentation Strategy (54 → ~270+ images)

Apply augmentation **only to training images**, never validation or test. Each original image generates ~4–5 augmented versions:

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    transforms.RandomRotation(degrees=10),
    transforms.RandomPerspective(distortion_scale=0.2, p=0.3),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

> ⚠ Do NOT use horizontal flip if your scene has an asymmetric depth structure (e.g., a corridor turning left — flipping it makes the label wrong for TL/TR). If in doubt, skip horizontal flip.

Part 3 — Dataset Split

From your 54 images:

- **Train:** 38 images (~70%)
- **Val:** 8 images (~15%)
- **Test:** 8 images (~15%)

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv('labels.csv')
train_df, temp_df = train_test_split(df, test_size=0.30, random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.50, random_state=42)

train_df.to_csv('train.csv', index=False)
val_df.to_csv('val.csv', index=False)
test_df.to_csv('test.csv', index=False)
```

Part 4 — Complete Classification Pipeline (PyTorch)

```
import torch
import torch.nn as nn
from torchvision import models
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import pandas as pd

label_map = {'N': 0, 'M': 1, 'F': 2}

class DepthDataset(Dataset):
    def __init__(self, csv_path, img_dir, transform):
        self.df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(f"{self.img_dir}/{row['filename']}").convert('RGB')
        img = self.transform(img)
        labels = torch.tensor([label_map[row['TL']], label_map[row['TR']],
                               label_map[row['BL']], label_map[row['BR']]])
        return img, labels
```

```
# Model with 4 heads
class DepthCNN(nn.Module):
    def __init__(self):
        super().__init__()
        backbone = models.resnet18(pretrained=True)
        self.features = nn.Sequential(*list(backbone.children())[:-1])
        self.heads = nn.ModuleList([nn.Linear(512, 3) for _ in range(4)])

    def forward(self, x):
        x = self.features(x).squeeze(-1).squeeze(-1)
        return [head(x) for head in self.heads]

# Training loop
model = DepthCNN()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss()

for epoch in range(50):
    model.train()
    for imgs, labels in train_loader:
        outputs = model(imgs)
        loss = sum(criterion(outputs[i], labels[:, i]) for i in range(4))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Summary Workflow

1. **Label 54 images** → `labels.csv` (30–60 min)
2. **Split into train/val/test** → 3 CSV files (10 min)
3. **Write Dataset class** → loads image + 4 labels (30 min)
4. **Build ResNet18 + 4 heads** → copy code above (20 min)
5. **Train for 50 epochs** with augmented training data (1–2 hours)
6. **Evaluate** per-head accuracy on test set (30 min)
7. **Optional:** color overlay visualization (1 hour) [2]

⁂

this is one of the images i took for the dataset so how do i split it into 4 quadrants and how do i label it should i do it with roboflow or something like bounding box or something which is the best method for such small dataset of 70 images

Good image — this is actually a solid tabletop scene with clear depth structure. Let me show you exactly how to label this and the best method for 70 images.

How to Label This Specific Image

Looking at your image, here's how the 2×2 split maps to depth:

```
| TL               | TR               |
| Red speaker       | Clown figure +   |
| (mid-far)         | remote (far)     |
|-------------------|------------------|
| BL               | BR               |
| Keychain (near)   | Empty table      |
| Luffy figure      | (near-mid)       |
| (near)           |                  |
```

Label for this image:

```
TL = M   (speaker is mid-table depth)
TR = F   (clown and remote are at the back of the table)
BL = N   (keychain and Luffy figure are right in front)
BR = M   (empty table surface, middle area)
```

So your CSV row would be: `IMG_20260526_151018.jpg,M,F,N,M`

Best Labeling Method for 70 Images

Skip Roboflow entirely. Roboflow is for bounding box / segmentation annotation — that's overkill and the wrong tool for this task. You don't need bounding boxes. You need just 4 labels per image. The best method is a simple Python script or a spreadsheet.[13]

Option 1 — Python Script (Recommended, ~1 hour for all 70)

This is the cleanest approach. Run this script, it shows each image, and you press keys to assign labels:

```
import cv2
import pandas as pd
import os
```

```
img_dir = "images/"          # your image folder
output_csv = "labels.csv"
label_map = {ord('n'): 'N', ord('m'): 'M', ord('f'): 'F'}
regions = ['TL', 'TR', 'BL', 'BR']
results = []

images = sorted(os.listdir(img_dir))

for img_name in images:
    img = cv2.imread(os.path.join(img_dir, img_name))
    h, w = img.shape[:2]

    # Draw quadrant lines
    vis = img.copy()
    cv2.line(vis, (w//2, 0), (w//2, h), (0, 255, 0), 2)
    cv2.line(vis, (0, h//2), (w, h//2), (0, 255, 0), 2)
    cv2.putText(vis, "TL", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
    cv2.putText(vis, "TR", (w//2+20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
    cv2.putText(vis, "BL", (20, h//2+40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
    cv2.putText(vis, "BR", (w//2+20, h//2+40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0)

    labels = []
    for region in regions:
        show = vis.copy()
        cv2.putText(show, f"Label {region} - press N/M/F",
                    (10, h-20), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255,255,0), 2)
        cv2.imshow("Label Image", show)

        while True:
            key = cv2.waitKey(0)
            if key in label_map:
                labels.append(label_map[key])
                break

    results.append([img_name] + labels)
    print(f"Labeled: {img_name} → {labels}")

cv2.destroyAllWindows()
pd.DataFrame(results, columns=['filename','TL','TR','BL','BR']).to_csv(output_csv, i
print("Done! Saved to", output_csv)
```

How it works:

- Shows image with green quadrant lines drawn on it
- Prompts you region by region (TL → TR → BL → BR)
- Press `N`, `M`, or `F` on keyboard for each
- Auto-saves all labels to CSV when done

Option 2 — Spreadsheet (No Coding, Slower)

Open all images one by one in your file viewer, mentally split them, and fill a Google Sheet / Excel with columns: `filename, TL, TR, BL, BR`. Takes ~2–3 hours but requires no setup.

Your Depth Rule for Tabletop Scenes

Apply this **consistently** across all 70 images:[13]

Zone	Tabletop Definition
Near (N)	Front edge of table, objects in the bottom 1/3 of frame
Middle (M)	Center of table, mid-depth objects
Far (F)	Back of table, wall behind, objects near the back edge

What NOT to Do

- **Don't use Roboflow** — it's for bounding boxes, not grid labels[13]
- **Don't use LabelImg or CVAT** — same reason, wrong tool
- **Don't label randomly** — your depth rule must stay identical for every single image

The Python script above is genuinely the fastest approach for exactly this task. You'll finish all 70 images in under an hour.

⁂

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4

this is the location of the images first will you rename all the images using a python script

Here's your ready-to-run script. Download it above.

How to Run It

Step 1 — Download and place the script anywhere (Desktop is fine)

Step 2 — Open a terminal in your Task 4 folder:

- Press `Win + R`, type `cmd`, hit Enter
- Navigate to the folder:

```
cd "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Com
```

Step 3 — Run the script:

```
python rename_images.py
```

What the Script Does

- Finds all `.jpg`, `.jpeg`, `.png` files in your Task 4 folder
- Sorts them **by capture time** (modification date) so order is preserved
- Renames them to: `image_001.jpg`, `image_002.jpg` ... `image_054.jpg`
- **Automatically creates** `labels.csv` with all filenames listed and empty TL/TR/BL/BR columns ready for you to fill in

Output Preview

```
Found 54 images
  IMG_20260526_151018.jpg  →  image_001.jpg
  IMG_20260526_151025.jpg  →  image_002.jpg
  ...
Done! Renamed 54 files.

Created blank labels.csv at:
C:\Users\gbhan\...\Task 4\labels.csv
```

After running, open `labels.csv` in **Excel or Google Sheets** and fill in `N`, `M`, or `F` for each quadrant per image. That's your labeling step done in one go.

C:\Users\gbhan\AppData\Local\Programs\Python\Python314\python.exe: can't open file 'C:\\Users\\gbhan\\COLLEGE NOTES ARRANGING\\SECOND SEMESTER\\Image-Processing and Computer Vision\\TASKS\\Task 4\\rename_images.py': [Errno 2] No such file or directory

The script file isn't in your Task 4 folder — it's somewhere else on your system. Fix it in two ways:

Option 1 — Easiest Fix (Drag and Drop)

1. Find the downloaded `rename_images.py` file (check your **Downloads** folder)
2. **Drag and drop** it into your Task 4 folder:

```
C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Compute
```

3. Then run:

```
python rename_images.py
```

Option 2 — Run It From Wherever the Script Is

If it's in Downloads, just give the full path to the script:

```
python "C:\Users\gbhan\Downloads\rename_images.py"
```

Option 3 — Quickest Fix (No File Needed)

Just paste this directly into your terminal and press Enter — no script file needed at all:

```
python -c "
import os, csv
folder = r'C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing
valid = {'.jpg','.jpeg','.png','.JPG','.JPEG','.PNG'}
files = sorted([f for f in os.listdir(folder) if os.path.splitext(f)[^5_1] in valid]
renamed=[]
for i,old in enumerate(files,1):
    ext=os.path.splitext(old)[^5_1].lower()
    new=f'image_{i:03d}{ext}'
    if old!=new: os.rename(os.path.join(folder,old),os.path.join(folder,new)); rena
all_imgs=sorted([f for f in os.listdir(folder) if os.path.splitext(f)[^5_1] in {'.jp
with open(os.path.join(folder,'labels.csv'),'w',newline='') as f:
    w=csv.writer(f); w.writerow(['filename','TL','TR','BL','BR'])
    [w.writerow([img,'','','','']) for img in all_imgs]
print(f'Done! Renamed {len(renamed)} files. labels.csv created.')
"
```

This is the most direct — copy, paste, Enter. Done.[15]

⁂

name change done now lets get to the labelling task

Good, renaming is done! Now let's go straight into labeling. Here's the fastest way to get all 54 images labeled.

The Labeling Script (Run This Now)

Save this as `label_images.py` inside your **Task 4 folder** and run it. It shows each image with the quadrant grid drawn on it, and you just press keys:

```
import cv2
import pandas as pd
import os

folder = r"C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing
csv_path = os.path.join(folder, "labels.csv")

label_map = {ord('n'): 'N', ord('m'): 'M', ord('f'): 'F',
             ord('N'): 'N', ord('M'): 'M', ord('F'): 'F'}
regions = ['TL', 'TR', 'BL', 'BR']

images = sorted([f for f in os.listdir(folder) if f.endswith('.jpg') or f.endswith(
results = []

for idx, img_name in enumerate(images):
    img = cv2.imread(os.path.join(folder, img_name))
    if img is None:
        continue
    h, w = img.shape[:2]
    labels = []

    for region in regions:
        vis = img.copy()
        # Draw quadrant lines
        cv2.line(vis, (w//2, 0), (w//2, h), (0, 255, 0), 3)
        cv2.line(vis, (0, h//2), (w, h//2), (0, 255, 0), 3)
        # Highlight current region
        coords = {
            'TL': (0, 0, w//2, h//2),
            'TR': (w//2, 0, w, h//2),
            'BL': (0, h//2, w//2, h),
            'BR': (w//2, h//2, w, h)
        }
        x1, y1, x2, y2 = coords[region]
        overlay = vis.copy()
        cv2.rectangle(overlay, (x1, y1), (x2, y2), (0, 255, 255), -1)
        cv2.addWeighted(overlay, 0.25, vis, 0.75, 0, vis)

        # Labels
        cv2.putText(vis, "TL", (20, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255,255,255)
        cv2.putText(vis, "TR", (w//2+20, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255,255
        cv2.putText(vis, "BL", (20, h//2+50), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255,255
        cv2.putText(vis, "BR", (w//2+20, h//2+50), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (25
        cv2.putText(vis, f"[{idx+1}/{len(images)}] Label {region} → Press N / M / F",
                    (10, h-15), cv2.FONT_HERSHEY_SIMPLEX, 0.85, (0, 255, 255), 2)

        cv2.namedWindow("Labeler", cv2.WINDOW_NORMAL)
        cv2.resizeWindow("Labeler", 900, 600)
        cv2.imshow("Labeler", vis)
```

```
        while True:
            key = cv2.waitKey(0)
            if key in label_map:
                labels.append(label_map[key])
                break
            elif key == 27:  # ESC to quit and save progress
                cv2.destroyAllWindows()
                pd.DataFrame(results, columns=['filename','TL','TR','BL','BR']).to_cs
                print("Saved progress and exited.")
                exit()

    results.append([img_name] + labels)
    print(f"[{idx+1}/{len(images)}] {img_name} → {labels}")

cv2.destroyAllWindows()
pd.DataFrame(results, columns=['filename','TL','TR','BL','BR']).to_csv(csv_path, ind
print(f"\n✅ Done! All {len(results)} images labeled. Saved to labels.csv")
```

How to Use It

Key	Label	Meaning
N	Near	Foreground, front of table, close objects
M	Middle	Mid-table, intermediate depth
F	Far	Back of table, wall, distant area
ESC	Save & Quit	Progress saved — you can resume later

- The **highlighted yellow quadrant** shows which region you're currently labeling
- It goes TL → TR → BL → BR for each image automatically
- Takes about **45–60 minutes** for all 54 images

Your Depth Rule (Tabletop Setup)

Apply this **consistently** for every image — looking at your Luffy photo as the reference:

- **Near (N):** Objects in the bottom portion, front edge of the table (Luffy figure, keychain)
- **Middle (M):** Mid-table area, medium distance objects (speaker in your photo)
- **Far (F):** Back of table, objects near the wall (clown figure, remote)

Run it with:

```
cd "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Com
python label_images.py
```

⁂

i think before running all these we should create a venv and i am using python 3.14 and 3.10 choose which is best for this task and give me commands step by step

Good thinking — always use a venv. Here's the verdict and the full setup.

Use Python 3.10 — Not 3.14

Python 3.14 is brand new (still in development phases) and **PyTorch does not support it yet.** OpenCV and torchvision also lag behind on 3.14. You will hit broken package errors immediately.[26]

Python 3.10 = stable, fully supported by PyTorch, OpenCV, torchvision, and scikit-learn. Use it for everything in this project.

Step-by-Step Setup

Step 1 — Navigate to your Task 4 folder:

```
cd "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Com
```

Step 2 — Create the venv using Python 3.10 explicitly:

```
C:\Users\gbhan\AppData\Local\Programs\Python\Python310\python.exe -m venv venv
```

Step 3 — Activate the venv:

```
venv\Scripts\activate
```

Your terminal prompt will now show `(venv)` at the start — that means it's active.[27]

Step 4 — Upgrade pip first:

```
python -m pip install --upgrade pip
```

Step 5 — Install all required packages:

```
pip install opencv-python pandas scikit-learn torch torchvision pillow matplotlib
```

This installs everything needed for labeling, training, and visualization. Takes 2–5 minutes depending on your internet speed.

Step 6 — Verify everything installed correctly:

```
python -c "import cv2, torch, torchvision, pandas, sklearn; print('All packages OK')
```

If you see `All packages OK` — you're ready to go.

Every Time You Come Back

Every new terminal session, just activate the venv again before running any script:

```
cd "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Com
venv\Scripts\activate
```

Then run your scripts normally (`python label_images.py`, etc.)

⁂

done with labelling but little confused whther i did it right or wrong because different camera angles made it confusing as the objects are close with some minor distance variations. so whats next

Stop stressing about this. Here's the honest truth and what to do next.

Your Labeling is Fine — Here's Why

This task uses **relative depth, not metric depth.** The assignment says: *"Label according to the dominant depth impression in each region"* and *"If a region is ambiguous, define an internal decision rule and apply it consistently."* You don't need perfect labels — you need **consistent** ones. The model will learn from the pattern, not from physical precision.[36]

The confusion from different camera angles is actually **normal and expected** — real depth estimation models face this exact problem. Your dataset having that variation is a feature, not a bug.[37]

Do This Quick Sanity Check First

Before training, run this to catch obvious mistakes in your CSV:

```
import pandas as pd

df = pd.read_csv(r"C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Pro
```

```
print("Total images:", len(df))
print("\nLabel distribution:")
for col in ['TL', 'TR', 'BL', 'BR']:
    print(f"  {col}:", df[col].value_counts().to_dict())

print("\nAny missing labels?", df[['TL','TR','BL','BR']].isnull().any().any())
print("\nInvalid labels?", (~df[['TL','TR','BL','BR']].isin(['N','M','F'])).any().any
```

What to check:

- All 4 columns should have only `N`, `M`, `F` — no blanks or typos
- Distribution shouldn't be **all one class** (e.g., 90% Far) — if it is, your labeling rule was too rigid
- Ideally, all 3 labels appear in each column

What's Next — The Full Pipeline

You're at **Step 4 of 7**. Here's where you stand:

Step	Task	Status
1	Capture images	✅ Done (54 images)
2	Rename images	✅ Done
3	Setup venv + install packages	✅ Done
4	Label images → labels.csv	✅ Done
5	Split dataset (train/val/test) + Augmentation	⬅ Next
6	Build & train ResNet18 model	Pending
7	Evaluate + optional visualization overlay	Pending

Tell me when you're ready and I'll give you the complete Step 5 code — dataset split + augmentation pipeline in one script, ready to run in your venv.

⁂

(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python check.py

Total images: 70

Label distribution:
TL: {'M': 37, 'N': 22, 'F': 11}

TR: {'M': 40, 'N': 21, 'F': 9}
BL: {'N': 66, 'M': 4}
BR: {'N': 68, 'M': 2}

Any missing labels? False

Invalid labels? False
this is the output of the above program whats next

Good output — but there's a serious problem you need to fix before training.

⚠ Critical Problem — BL and BR Have No "F" Labels

Look at your distribution:

```
BL: {'N': 66, 'M': 4}   ← zero F labels
BR: {'N': 68, 'M': 2}   ← zero F labels
```

This is bad. The model will never learn to predict `Far` for the bottom quadrants because it has never seen that label there. Your bottom two heads will basically just output `N` every time for every image — that's not depth estimation, that's memorization.[52]

This makes sense physically — in a tabletop scene, the bottom of the image is always the front of the table (Near). But it means your dataset lacks variety for BL/BR.

Two Options — Pick One

Option A — Re-label Some Images (Recommended, 20 min)

Go back to images where the camera angle was tilted higher or where the bottom of the image shows mid-table or background. Relabel those BL/BR as `M` or `F`. You only need about **10–15 images** with non-N labels in BL/BR to make a difference.

Option B — Accept It and Proceed (Faster)

The assignment doesn't require perfectly balanced classes. You can proceed but must use **class weights** in your loss function to compensate. This tells the model to penalize mistakes on rare classes more heavily.[52]

Regardless — Proceed With Dataset Split Now

Run this script `split_dataset.py`:

```
import pandas as pd
from sklearn.model_selection import train_test_split
import os
```

```
folder = r"C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing
df = pd.read_csv(os.path.join(folder, "labels.csv"))

# 70/15/15 split
train_df, temp_df = train_test_split(df, test_size=0.30, random_state=42, shuffle=Tr
val_df, test_df   = train_test_split(temp_df, test_size=0.50, random_state=42)

train_df.to_csv(os.path.join(folder, "train.csv"), index=False)
val_df.to_csv(os.path.join(folder,   "val.csv"),   index=False)
test_df.to_csv(os.path.join(folder,  "test.csv"),  index=False)

print(f"Train: {len(train_df)} | Val: {len(val_df)} | Test: {len(test_df)}")
print("\nTrain label distribution:")
for col in ['TL','TR','BL','BR']:
    print(f"  {col}:", train_df[col].value_counts().to_dict())
```

Run it:

```
python split_dataset.py
```

What To Do After Split

Tell me which option you pick (A or B) and I'll give you the full training script with the correct loss setup. If you go with Option B I'll add class weights automatically — just say the word.

⁂

can't we increase the quadrant number to 6 quadrants instead of 4 as the image is not being split perfectly as there are very less far sections what do you think

Straight answer — **do not change to 6 quadrants.** Here's why:

Why 6 Quadrants is a Bad Idea

The assignment explicitly specifies a 2×2 grid. Changing this means you're deviating from the task requirements — Prof. Schaffer defined the grid structure and the evaluation will be based on it. Don't risk marks over this.[63]

More importantly, 6 quadrants is not your actual problem. Your real problem is this:

```
BL: {'N': 66, 'M': 4}  ← 0 Far
BR: {'N': 68, 'M': 2}  ← 0 Far
```

More quadrants won't fix an imbalanced dataset — they'll make it **worse** because now you have even more regions with zero Far labels and a smaller dataset split across more outputs.

The Real Fix — Use Class Weights

This is the standard professional solution for imbalanced labels. You don't re-label, you don't change the grid — you just tell the loss function to penalize mistakes on rare classes harder. [63]

For BL and BR, since `F` never appears, the model will just learn to always predict `N`. Class weights compensate for this:

```
from torch.nn import CrossEntropyLoss
import torch

# Weights are inverse of frequency
# TL: N=22, M=37, F=11  → weights: [37/22, 37/37, 37/11] ≈ [1.7, 1.0, 3.4]
# TR: N=21, M=40, F=9   → weights: [40/21, 40/40, 40/9]  ≈ [1.9, 1.0, 4.4]
# BL: N=66, M=4, F=0    → treat F as rare: [1.0, 16.5, 5.0] (manual boost)
# BR: N=68, M=2, F=0    → treat F as rare: [1.0, 34.0, 5.0] (manual boost)

weights = {
    'TL': torch.tensor([1.7, 1.0, 3.4]),
    'TR': torch.tensor([1.9, 1.0, 4.4]),
    'BL': torch.tensor([1.0, 16.5, 5.0]),
    'BR': torch.tensor([1.0, 34.0, 5.0]),
}

criterion_TL = CrossEntropyLoss(weight=weights['TL'])
criterion_TR = CrossEntropyLoss(weight=weights['TR'])
criterion_BL = CrossEntropyLoss(weight=weights['BL'])
criterion_BR = CrossEntropyLoss(weight=weights['BR'])
```

My Honest Advice

Your labeling actually reflects **reality** — in a tabletop scene, the bottom of the image IS almost always Near. That's a valid depth pattern your model should learn. The assignment only asks for relative depth structure, and your data has that clearly in TL/TR. [63]

Run the dataset split now and move to training. I'll build the full training script with class weights already baked in. Say go.

⁂

after this output

venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python check.py
Total images: 70

Label distribution:
TL: {'M': 37, 'N': 22, 'F': 11}
TR: {'M': 40, 'N': 21, 'F': 9}
BL: {'N': 66, 'M': 4}
BR: {'N': 68, 'M': 2}

Any missing labels? False

Invalid labels? False

i didnot make any changes to the labelling so should i split the data set now using the code you gave

Yes, go ahead — run the split script exactly as I gave it. No changes needed.

Just copy-paste this into a new file `split_dataset.py` and run it:

```
import pandas as pd
from sklearn.model_selection import train_test_split
import os

folder = r"C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing
df = pd.read_csv(os.path.join(folder, "labels.csv"))

train_df, temp_df = train_test_split(df, test_size=0.30, random_state=42, shuffle=Tr
val_df, test_df   = train_test_split(temp_df, test_size=0.50, random_state=42)

train_df.to_csv(os.path.join(folder, "train.csv"), index=False)
val_df.to_csv(os.path.join(folder,   "val.csv"),   index=False)
test_df.to_csv(os.path.join(folder,  "test.csv"),  index=False)

print(f"Train: {len(train_df)} | Val: {len(val_df)} | Test: {len(test_df)}")
print("\nTrain label distribution:")
for col in ['TL','TR','BL','BR']:
    print(f"  {col}:", train_df[col].value_counts().to_dict())
```

Then run:

```
python split_dataset.py
```

Expected output:

```
Train: 49 | Val: 10 | Test: 11
```

Once you paste that output here, I'll give you the **complete training script** — ResNet18 + 4 heads + class weights + augmentation — all in one file, ready to run. That's the big step next.

⁂

(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python .\split_dataset.py

Train: 49 | Val: 10 | Test: 11

Train label distribution:
TL: {'M': 23, 'N': 17, 'F': 9}
TR: {'M': 24, 'N': 17, 'F': 8}
BL: {'N': 47, 'M': 2}
BR: {'N': 48, 'M': 1}

while giving me the code for training make sure to use gpu qas i have rtx 4060 in my laptop with 6gb vram so the training runs fastly

Download `train.py` above and drop it into your Task 4 folder. Then run:

```
python train.py
```

What This Script Does

GPU Setup:

- Auto-detects your RTX 4060 via `torch.cuda.is_available()` [82]
- Uses **mixed precision (AMP)** — this cuts VRAM usage in half and makes training ~2x faster on your RTX 4060's Tensor Cores
- `pin_memory=True` speeds up CPU→GPU data transfer

Model Architecture:

- ResNet18 pretrained backbone + 4 independent heads (TL, TR, BL, BR) [82]
- Each head has `Linear(512→128) → ReLU → Dropout(0.3) → Linear(128→3)` — the dropout prevents overfitting on your small dataset

Training Features:

- Class weights baked in for all 4 heads — compensates for your BL/BR imbalance
- Cosine annealing LR scheduler — gradually reduces learning rate for better convergence
- Saves best model automatically as `best_model.pth`

- Prints a clean table every 5 epochs

Expected output on your RTX 4060:

```
Using device: cuda
GPU: NVIDIA GeForce RTX 4060 Laptop GPU
VRAM: 8.0 GB

=================================================
 Epoch | Train Loss | Train Acc |  Val Acc
=================================================
     1 |     1.2341 |    0.5123 |  0.5000
     5 |     0.9821 |    0.6234 |  0.5500 ← best
    10 |     0.7654 |    0.7100 |  0.6250 ← best
    ...
```

Training 60 epochs should finish in **under 3 minutes** on your GPU. Once it's done, paste the test evaluation output here and we'll move to the visualization overlay.

⁂

(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python .\split_dataset.py

Train: 49 | Val: 10 | Test: 11

Train label distribution:
TL: {'M': 23, 'N': 17, 'F': 9}
TR: {'M': 24, 'N': 17, 'F': 8}
BL: {'N': 47, 'M': 2}
BR: {'N': 48, 'M': 1}
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python train.py
Using device: cpu
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to C:\Users\gbhan/.cache\torch\hub\checkpoints\resnet18-f37072fd.pth
100.0%
C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py:106: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
scaler = torch.cuda.amp.GradScaler(enabled=(DEVICE.type == "cuda"))

==========

==========

Epoch | Train Loss | Train Acc | Val Acc

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py:1095: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
Using device: cpu
Using device: cpu
C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py:106: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
scaler = torch.cuda.amp.GradScaler(enabled=(DEVICE.type == "cuda"))

==========

==========

Epoch | Train Loss | Train Acc | Val Acc

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py:1095: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
Traceback (most recent call last):

```
  File "<string>", line 1, in <module>
```

File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 116, in spawn_main
exitcode = _main(fd, parent_sentinel)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 125, in _main
prepare(preparation_data)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 236, in prepare
_fixup_main_from_path(data['init_main_from_path'])
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 287, in _fixup_main_from_path
main_content = runpy.run_path(main_path,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 289, in run_path

return _run_module_code(code, init_globals, run_name,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 96, in _run_module_code
_run_code(code, mod_globals, init_globals,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 86, in _run_code
exec(code, run_globals)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py", line 139, in <module>
for imgs, labels in train_loader:
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 498, in iter
return self._get_iterator()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 431, in _get_iterator
return _MultiProcessingDataLoaderIter(self)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1169, in init
w.start()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\process.py", line 121, in start
self._popen = self._Popen(self)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 224, in _Popen
return _default_context.get_context().Process._Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 336, in _Popen
return Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\popen_spawn_win32.py", line 45, in init
prep_data = spawn.get_preparation_data(process_obj._name)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 154, in get_preparation_data
_check_not_importing_main()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 134, in _check_not_importing_main
raise RuntimeError('''
RuntimeError:
An attempt has been made to start a new process before the
current process has finished its bootstrapping phase.

```
    This probably means that you are not using fork to start your
    child processes and you have forgotten to use the proper idiom
    in the main module:

        if __name__ == '__main__':
            freeze_support()
            ...

    The "freeze_support()" line can be omitted if the program
    is not going to be frozen to produce an executable.
```

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py:106: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
scaler = torch.cuda.amp.GradScaler(enabled=(DEVICE.type == "cuda"))

==========

==========

Epoch | Train Loss | Train Acc | Val Acc

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py:1095: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
Traceback (most recent call last):

```
  File "<string>", line 1, in <module>
```

File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 116, in spawn_main
exitcode = _main(fd, parent_sentinel)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 125, in _main
prepare(preparation_data)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 236, in prepare
_fixup_main_from_path(data['init_main_from_path'])
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 287, in _fixup_main_from_path
main_content = runpy.run_path(main_path,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 289, in run_path
return _run_module_code(code, init_globals, run_name,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 96, in

_run_module_code
_run_code(code, mod_globals, init_globals,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 86, in _run_code
exec(code, run_globals)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py", line 139, in <module>
for imgs, labels in train_loader:
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 498, in iter
return self._get_iterator()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 431, in _get_iterator
return _MultiProcessingDataLoaderIter(self)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1169, in init
w.start()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\process.py", line 121, in start
self._popen = self._Popen(self)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 224, in _Popen
return _default_context.get_context().Process._Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 336, in _Popen
return Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\popen_spawn_win32.py", line 45, in init
prep_data = spawn.get_preparation_data(process_obj._name)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 154, in get_preparation_data
_check_not_importing_main()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 134, in _check_not_importing_main
raise RuntimeError('''
RuntimeError:
An attempt has been made to start a new process before the
current process has finished its bootstrapping phase.

```
    This probably means that you are not using fork to start your
    child processes and you have forgotten to use the proper idiom
    in the main module:
```

```
        if __name__ == '__main__':
            freeze_support()
            ...

    The "freeze_support()" line can be omitted if the program
    is not going to be frozen to produce an executable.
```

Traceback (most recent call last):
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1287, in _try_get_data
data = self._data_queue.get(timeout=timeout)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\queues.py", line 114, in get
raise Empty
_queue.Empty

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py", line 139, in <module>
for imgs, labels in train_loader:
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 718, in next
data = self._next_data()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1501, in _next_data
idx, data = self._get_data()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1460, in _get_data
success, data = self._try_get_data()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1300, in _try_get_data
raise RuntimeError(
RuntimeError: DataLoader worker (pid(s) 21812, 25848) exited unexpectedly
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4>

You hit **two separate problems**, and both are fixable:

1. **Your PyTorch install is CPU-only**, so it ignored your RTX 4060 and ran on CPU. A CUDA-enabled PyTorch build is required for GPU training.[83] [84]

2. On Windows, PyTorch `DataLoader` with worker processes often needs the code protected by `if __name__ == "__main__":`, and using `num_workers=0` is the safest fix.[85] [86]

Fix CUDA first

Run these commands inside your activated venv to replace the CPU build with a CUDA build; PyTorch provides pip wheels with bundled CUDA runtime, so you do not need a separate local CUDA toolkit just to train.[84] [83]

```
pip uninstall -y torch torchvision torchaudio
pip cache purge
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/wh
python -c "import torch; print('cuda:', torch.cuda.is_available()); print('device:'
```

If that last command does **not** print `cuda: True`, then the issue is not your code — it is your driver / install stack, and forcing training now is pointless.[83] [84]

Fix the training script

Your current crash is the classic Windows multiprocessing issue. The correct move is:

- keep everything inside `if __name__ == "__main__":`
- set `num_workers=0`
- disable `pin_memory` when CUDA is not available [86] [85]

Open `train.py` and make these exact edits:

1) Change DataLoaders to this

```
train_loader = DataLoader(train_ds, batch_size=BATCH, shuffle=True, num_workers=0, p
val_loader   = DataLoader(val_ds,   batch_size=BATCH, shuffle=False, num_workers=0, p
test_loader  = DataLoader(test_ds,  batch_size=BATCH, shuffle=False, num_workers=0, p
```

2) Make sure training starts only here

```
if __name__ == "__main__":
    main()
```

3) Put the actual training code inside `main()`

That means model creation, dataset creation, dataloaders, optimizer, scheduler, and loops all belong inside `main()`.[85]

What to do now

Follow this order exactly:

1. Reinstall CUDA PyTorch.[84] [83]
2. Verify `torch.cuda.is_available()` returns `True`.[83]
3. Edit `train.py` with `num_workers=0` and proper `main()` guard.[86]
4. Run:

```
python train.py
```

Brutal assessment

Your dataset is already small and imbalanced. Training it on CPU while fighting broken Windows multiprocessing is wasted time. Fix the environment first, then train once properly. Otherwise you're just producing messy results slower.[87]

Paste the output of this command next:

```
python -c "import torch; print(torch.cuda.is_available()); print(torch.version.cuda
```

⁂

PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> & C:\Users\gbhan\AppData\Roaming\uv\python\cpython-3.14.3-windows-x86_64-none\python.exe "c:/Users/gbhan/COLLEGE NOTES ARRANGING/SECOND SEMESTER/Image-Processing and Computer Vision/TASKS/Task 4/label_images.py"

Traceback (most recent call last):
File "c:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\label_images.py", line 1, in <module>
import cv2
ModuleNotFoundError: No module named 'cv2'
PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> cd "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4"
PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing

and Computer Vision\TASKS\Task 4>
C:\Users\gbhan\AppData\Local\Programs\Python\Python310\python.exe -m venv venv
PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> venv\Scripts\activate
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\gbhan\college notes arranging\second semester\image-processing and computer vision\tasks\task 4\venv\lib\site-packages (23.0.1)
Collecting pip
Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
━━ 1.8/1.8 MB 16.5 MB/s eta 0:00:00
Installing collected packages: pip
Attempting uninstall: pip
Found existing installation: pip 23.0.1
Uninstalling pip-23.0.1:
Successfully uninstalled pip-23.0.1
Successfully installed pip-26.1.1
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python -m pip install --upgrade pip
Requirement already satisfied: pip in .\venv\lib\site-packages (26.1.1)
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> venv\Scripts\activate
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> deactivate
PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> venv\Scripts\activate
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> pip install opencv-python pandas scikit-learn torch torchvision pillow matplotlib
Collecting opencv-python
Using cached opencv_python-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Collecting pandas
Using cached pandas-2.3.3-cp310-cp310-win_amd64.whl.metadata (19 kB)
Collecting scikit-learn
Using cached scikit_learn-1.7.2-cp310-cp310-win_amd64.whl.metadata (11 kB)
Collecting torch
Downloading torch-2.12.0-cp310-cp310-win_amd64.whl.metadata (31 kB)
Collecting torchvision
Downloading torchvision-0.27.0-cp310-cp310-win_amd64.whl.metadata (5.5 kB)
Collecting pillow
Using cached pillow-12.2.0-cp310-cp310-win_amd64.whl.metadata (9.0 kB)
Collecting matplotlib
Downloading matplotlib-3.10.9-cp310-cp310-win_amd64.whl.metadata (52 kB)
Collecting numpy>=2 (from opencv-python)
Using cached numpy-2.2.6-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting python-dateutil>=2.8.2 (from pandas)

Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
Downloading pytz-2026.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
Downloading tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting scipy>=1.8.0 (from scikit-learn)
Using cached scipy-1.15.3-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting filelock (from torch)
Downloading filelock-3.29.0-py3-none-any.whl.metadata (2.0 kB)
Collecting typing-extensions>=4.10.0 (from torch)
Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: setuptools<82 in .\venv\lib\site-packages (from torch) (65.5.0)
Collecting sympy>=1.13.3 (from torch)
Using cached sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx>=2.5.1 (from torch)
Downloading networkx-3.4.2-py3-none-any.whl.metadata (6.3 kB)
Collecting jinja2 (from torch)
Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting fsspec>=0.8.5 (from torch)
Downloading fsspec-2026.4.0-py3-none-any.whl.metadata (10 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
Using cached contourpy-1.3.2-cp310-cp310-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
Downloading fonttools-4.63.0-cp310-cp310-win_amd64.whl.metadata (121 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
Using cached kiwisolver-1.5.0-cp310-cp310-win_amd64.whl.metadata (5.2 kB)
Collecting packaging>=20.0 (from matplotlib)
Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting pyparsing>=3 (from matplotlib)
Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch)
Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Collecting MarkupSafe>=2.0 (from jinja2->torch)
Downloading markupsafe-3.0.3-cp310-cp310-win_amd64.whl.metadata (2.8 kB)
Using cached opencv_python-4.13.0.92-cp37-abi3-win_amd64.whl (40.2 MB)
Using cached pandas-2.3.3-cp310-cp310-win_amd64.whl (11.3 MB)
Using cached scikit_learn-1.7.2-cp310-cp310-win_amd64.whl (8.9 MB)
Downloading torch-2.12.0-cp310-cp310-win_amd64.whl (122.9 MB)

━━ 122.9/122.9 MB 43.6 MB/s 0:00:02
Downloading torchvision-0.27.0-cp310-cp310-win_amd64.whl (3.4 MB)
━━ 3.4/3.4 MB 50.4 MB/s 0:00:00
Using cached pillow-12.2.0-cp310-cp310-win_amd64.whl (7.1 MB)
Downloading matplotlib-3.10.9-cp310-cp310-win_amd64.whl (8.2 MB)
━━ 8.2/8.2 MB 50.6 MB/s 0:00:00
Using cached contourpy-1.3.2-cp310-cp310-win_amd64.whl (221 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.63.0-cp310-cp310-win_amd64.whl (1.6 MB)
━━ 1.6/1.6 MB 42.1 MB/s 0:00:00
Downloading fsspec-2026.4.0-py3-none-any.whl (203 kB)
Using cached joblib-1.5.3-py3-none-any.whl (309 kB)
Using cached kiwisolver-1.5.0-cp310-cp310-win_amd64.whl (73 kB)
Downloading networkx-3.4.2-py3-none-any.whl (1.7 MB)
━━ 1.7/1.7 MB 31.1 MB/s 0:00:00
Using cached numpy-2.2.6-cp310-cp310-win_amd64.whl (12.9 MB)
Downloading packaging-26.2-py3-none-any.whl (100 kB)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading pytz-2026.2-py2.py3-none-any.whl (510 kB)
Using cached scipy-1.15.3-cp310-cp310-win_amd64.whl (41.3 MB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Using cached sympy-1.14.0-py3-none-any.whl (6.3 MB)
Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Downloading filelock-3.29.0-py3-none-any.whl (39 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp310-cp310-win_amd64.whl (15 kB)
Installing collected packages: pytz, mpmath, tzdata, typing-extensions, threadpoolctl, sympy, six, pyparsing, pillow, packaging, numpy, networkx, MarkupSafe, kiwisolver, joblib, fsspec, fonttools, filelock, cycler, scipy, python-dateutil, opencv-python, jinja2, contourpy, torch, scikit-learn, pandas, matplotlib, torchvision
Successfully installed MarkupSafe-3.0.3 contourpy-1.3.2 cycler-0.12.1 filelock-3.29.0 fonttools-4.63.0 fsspec-2026.4.0 jinja2-3.1.6 joblib-1.5.3 kiwisolver-1.5.0 matplotlib-3.10.9 mpmath-1.3.0 networkx-3.4.2 numpy-2.2.6 opencv-python-4.13.0.92 packaging-26.2 pandas-2.3.3 pillow-12.2.0 pyparsing-3.3.2 python-dateutil-2.9.0.post0 pytz-2026.2 scikit-learn-1.7.2 scipy-1.15.3 six-1.17.0 sympy-1.14.0 threadpoolctl-3.6.0 torch-2.12.0 torchvision-0.27.0 typing-extensions-4.15.0 tzdata-2026.2
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python -c "import cv2, torch, torchvision, pandas, sklearn; print('All packages OK')"
All packages OK
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python .\label_images.py

[1/70] image_001.jpg → ['N', 'F', 'N', 'N']
[2/70] image_002.jpg → ['M', 'F', 'N', 'N']
[3/70] image_003.jpg → ['M', 'F', 'N', 'N']
[4/70] image_004.jpg → ['M', 'F', 'N', 'N']
[5/70] image_005.jpg → ['M', 'M', 'N', 'N']
[6/70] image_006.jpg → ['M', 'M', 'N', 'N']
[7/70] image_007.jpg → ['F', 'M', 'N', 'N']
[8/70] image_008.jpg → ['M', 'M', 'N', 'N']
[9/70] image_009.jpg → ['F', 'N', 'N', 'N']
[10/70] image_010.jpg → ['M', 'M', 'N', 'N']
[11/70] image_011.jpg → ['M', 'M', 'N', 'N']
[12/70] image_012.jpg → ['F', 'F', 'N', 'N']
[13/70] image_013.jpg → ['F', 'M', 'M', 'N']
[14/70] image_014.jpg → ['F', 'F', 'M', 'N']
[15/70] image_015.jpg → ['F', 'M', 'M', 'N']
[16/70] image_016.jpg → ['F', 'F', 'N', 'N']
[17/70] image_017.jpg → ['M', 'M', 'N', 'N']
[18/70] image_018.jpg → ['F', 'F', 'N', 'N']
[19/70] image_019.jpg → ['M', 'M', 'N', 'N']
[20/70] image_020.jpg → ['M', 'M', 'N', 'N']
[21/70] image_021.jpg → ['M', 'M', 'N', 'N']
[22/70] image_022.jpg → ['F', 'F', 'N', 'N']
[23/70] image_023.jpg → ['M', 'M', 'N', 'N']
[24/70] image_024.jpg → ['M', 'M', 'N', 'N']
[25/70] image_025.jpg → ['M', 'M', 'N', 'N']
[26/70] image_026.jpg → ['M', 'M', 'N', 'N']
[27/70] image_027.jpg → ['M', 'M', 'N', 'N']
[28/70] image_028.jpg → ['M', 'M', 'N', 'N']
[29/70] image_029.jpg → ['M', 'M', 'N', 'N']
[30/70] image_030.jpg → ['M', 'M', 'N', 'N']
[31/70] image_031.jpg → ['M', 'M', 'N', 'N']
[32/70] image_032.jpg → ['M', 'M', 'N', 'N']
[33/70] image_033.jpg → ['N', 'N', 'N', 'N']
[34/70] image_034.jpg → ['N', 'N', 'N', 'N']
[35/70] image_035.jpg → ['N', 'M', 'N', 'M']
[36/70] image_036.jpg → ['M', 'M', 'N', 'N']
[37/70] image_037.jpg → ['M', 'M', 'N', 'N']
[38/70] image_038.jpg → ['N', 'N', 'N', 'N']
[39/70] image_039.jpg → ['M', 'M', 'N', 'N']
[40/70] image_040.jpg → ['N', 'N', 'N', 'N']
[41/70] image_041.jpg → ['N', 'N', 'N', 'N']
[42/70] image_042.jpg → ['N', 'N', 'N', 'N']
[43/70] image_043.jpg → ['N', 'N', 'N', 'N']
[44/70] image_044.jpg → ['N', 'N', 'N', 'N']
[45/70] image_045.jpg → ['M', 'M', 'N', 'N']
[46/70] image_046.jpg → ['M', 'M', 'N', 'N']

[47/70] image_047.jpg → ['M', 'M', 'N', 'N']
[48/70] image_048.jpg → ['M', 'M', 'N', 'N']
[49/70] image_049.jpg → ['F', 'M', 'N', 'N']
[50/70] image_050.jpg → ['F', 'M', 'N', 'N']
[51/70] image_051.jpg → ['M', 'M', 'N', 'N']
[52/70] image_052.jpg → ['M', 'M', 'N', 'N']
[53/70] image_053.jpg → ['M', 'M', 'N', 'N']
[54/70] image_054.jpg → ['M', 'M', 'N', 'N']
[55/70] image_055.jpg → ['M', 'M', 'M', 'M']
[56/70] image_056.jpg → ['M', 'M', 'N', 'N']
[57/70] image_057.jpg → ['M', 'M', 'N', 'N']
[58/70] image_058.jpg → ['M', 'M', 'N', 'N']
[59/70] image_059.jpg → ['N', 'N', 'N', 'N']
[60/70] image_060.jpg → ['N', 'N', 'N', 'N']
[61/70] image_061.jpg → ['N', 'N', 'N', 'N']
[62/70] image_062.jpg → ['N', 'N', 'N', 'N']
[63/70] image_063.jpg → ['N', 'N', 'N', 'N']
[64/70] image_064.jpg → ['N', 'N', 'N', 'N']
[65/70] image_065.jpg → ['N', 'N', 'N', 'N']
[66/70] image_066.jpg → ['N', 'N', 'N', 'N']
[67/70] image_067.jpg → ['N', 'N', 'N', 'N']
[68/70] image_068.jpg → ['N', 'N', 'N', 'N']
[69/70] image_069.jpg → ['N', 'N', 'N', 'N']
[70/70] image_070.jpg → ['N', 'N', 'N', 'N']

✅ Done! All 70 images labeled. Saved to labels.csv
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python check.py
Total images: 70

Label distribution:
TL: {'M': 37, 'N': 22, 'F': 11}
TR: {'M': 40, 'N': 21, 'F': 9}
BL: {'N': 66, 'M': 4}
BR: {'N': 68, 'M': 2}

Any missing labels? False

Invalid labels? False
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python .\split_dataset.py
Train: 49 | Val: 10 | Test: 11

Train label distribution:
TL: {'M': 23, 'N': 17, 'F': 9}
TR: {'M': 24, 'N': 17, 'F': 8}
BL: {'N': 47, 'M': 2}
BR: {'N': 48, 'M': 1}

(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python train.py
Using device: cpu
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to C:\Users\gbhan/.cache\torch\hub\checkpoints\resnet18-f37072fd.pth
100.0%
C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py:106: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
scaler = torch.cuda.amp.GradScaler(enabled=(DEVICE.type == "cuda"))

==========

==========

Epoch | Train Loss | Train Acc | Val Acc

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py:1095: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
Using device: cpu
Using device: cpu
C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py:106: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
scaler = torch.cuda.amp.GradScaler(enabled=(DEVICE.type == "cuda"))

==========

==========

Epoch | Train Loss | Train Acc | Val Acc

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py:1095: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
Traceback (most recent call last):

```
  File "<string>", line 1, in <module>
```

File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 116, in spawn_main

exitcode = _main(fd, parent_sentinel)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 125, in _main
prepare(preparation_data)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 236, in prepare
_fixup_main_from_path(data['init_main_from_path'])
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 287, in _fixup_main_from_path
main_content = runpy.run_path(main_path,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 289, in run_path
return _run_module_code(code, init_globals, run_name,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 96, in _run_module_code
_run_code(code, mod_globals, init_globals,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 86, in _run_code
exec(code, run_globals)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py", line 139, in <module>
for imgs, labels in train_loader:
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 498, in iter
return self._get_iterator()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 431, in _get_iterator
return _MultiProcessingDataLoaderIter(self)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1169, in init
w.start()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\process.py", line 121, in start
self._popen = self._Popen(self)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 224, in _Popen
return _default_context.get_context().Process._Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 336, in _Popen
return Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\popen_spa

wn_win32.py", line 45, in init
prep_data = spawn.get_preparation_data(process_obj._name)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 154, in get_preparation_data
_check_not_importing_main()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 134, in _check_not_importing_main
raise RuntimeError('''
RuntimeError:
An attempt has been made to start a new process before the
current process has finished its bootstrapping phase.

```
    This probably means that you are not using fork to start your
    child processes and you have forgotten to use the proper idiom
    in the main module:

        if __name__ == '__main__':
            freeze_support()
            ...

    The "freeze_support()" line can be omitted if the program
    is not going to be frozen to produce an executable.
```

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py:106: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
scaler = torch.cuda.amp.GradScaler(enabled=(DEVICE.type == "cuda"))

==========

==========

Epoch | Train Loss | Train Acc | Val Acc

C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py:1095: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().init(loader)
Traceback (most recent call last):

```
  File "<string>", line 1, in <module>
```

File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 116, in spawn_main
exitcode = _main(fd, parent_sentinel)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\

spawn.py", line 125, in _main
prepare(preparation_data)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 236, in prepare
_fixup_main_from_path(data['init_main_from_path'])
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 287, in _fixup_main_from_path
main_content = runpy.run_path(main_path,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 289, in run_path
return _run_module_code(code, init_globals, run_name,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 96, in _run_module_code
_run_code(code, mod_globals, init_globals,
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\runpy.py", line 86, in _run_code
exec(code, run_globals)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py", line 139, in <module>
for imgs, labels in train_loader:
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 498, in iter
return self._get_iterator()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 431, in _get_iterator
return _MultiProcessingDataLoaderIter(self)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1169, in init
w.start()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\process.py", line 121, in start
self._popen = self._Popen(self)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 224, in _Popen
return _default_context.get_context().Process._Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\context.py", line 336, in _Popen
return Popen(process_obj)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\popen_spawn_win32.py", line 45, in init
prep_data = spawn.get_preparation_data(process_obj._name)

File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 154, in get_preparation_data
_check_not_importing_main()
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\spawn.py", line 134, in _check_not_importing_main
raise RuntimeError('''
RuntimeError:
An attempt has been made to start a new process before the
current process has finished its bootstrapping phase.

```
    This probably means that you are not using fork to start your
    child processes and you have forgotten to use the proper idiom
    in the main module:

        if __name__ == '__main__':
            freeze_support()
            ...

    The "freeze_support()" line can be omitted if the program
    is not going to be frozen to produce an executable.
```

Traceback (most recent call last):
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1287, in _try_get_data
data = self._data_queue.get(timeout=timeout)
File "C:\Users\gbhan\AppData\Local\Programs\Python\Python310\lib\multiprocessing\queues.py", line 114, in get
raise Empty
_queue.Empty

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py", line 139, in <module>
for imgs, labels in train_loader:
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 718, in next
data = self._next_data()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1501, in _next_data
idx, data = self._get_data()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1460, in _get_data

success, data = self._try_get_data()
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\torch\utils\data\dataloader.py", line 1300, in _try_get_data
raise RuntimeError(
RuntimeError: DataLoader worker (pid(s) 21812, 25848) exited unexpectedly
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> pip uninstall -y torch torchvision torchaudio
Found existing installation: torch 2.12.0
Uninstalling torch-2.12.0:
Successfully uninstalled torch-2.12.0
Found existing installation: torchvision 0.27.0
Uninstalling torchvision-0.27.0:
Successfully uninstalled torchvision-0.27.0
WARNING: Skipping torchaudio as it is not installed.
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> pip cache purge
Files removed: 816 (1446.4 MB)
Directories removed: 423
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
Looking in indexes: https://download.pytorch.org/whl/cu121
Collecting torch
Downloading torch-2.5.1%2Bcu121-cp310-cp310-win_amd64.whl (2449.4 MB)
━━ 2.4/2.4 GB 16.0 MB/s 0:00:45
Collecting torchvision
Downloading torchvision-0.20.1%2Bcu121-cp310-cp310-win_amd64.whl (6.1 MB)
━━ 6.1/6.1 MB 30.9 MB/s 0:00:00
Collecting torchaudio
Downloading torchaudio-2.5.1%2Bcu121-cp310-cp310-win_amd64.whl (4.1 MB)
━━ 4.1/4.1 MB 41.2 MB/s 0:00:00
Requirement already satisfied: filelock in .\venv\lib\site-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.8.0 in .\venv\lib\site-packages (from torch) (4.15.0)
Requirement already satisfied: networkx in .\venv\lib\site-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in .\venv\lib\site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in .\venv\lib\site-packages (from torch) (2026.4.0)
Collecting sympy1.13.1 (from torch)
Downloading sympy-1.13.1-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in .\venv\lib\site-packages (from sympy1.13.1->torch) (1.3.0)
Requirement already satisfied: numpy in .\venv\lib\site-packages (from torchvision) (2.2.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in .\venv\lib\site-packages (from torchvision) (12.2.0)
Requirement already satisfied: MarkupSafe>=2.0 in .\venv\lib\site-packages (from jinja2-

>torch) (3.0.3)
Downloading sympy-1.13.1-py3-none-any.whl (6.2 MB)
━━ 6.2/6.2 MB 31.7 MB/s 0:00:00
Installing collected packages: sympy, torch, torchvision, torchaudio
Attempting uninstall: sympy
Found existing installation: sympy 1.14.0
Uninstalling sympy-1.14.0:
Successfully uninstalled sympy-1.14.0
Successfully installed sympy-1.13.1 torch-2.5.1+cu121 torchaudio-2.5.1+cu121 torchvision-0.20.1+cu121
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python -c "import torch; print('cuda:', torch.cuda.is_available()); print('device:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU'); print('torch cuda version:', torch.version.cuda)"
cuda: True
device: NVIDIA GeForce RTX 4050 Laptop GPU
torch cuda version: 12.1
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4>

this is output till now and if you want to make any changes to the program

```
import os
import torch
import torch.nn as nn
from torchvision import models, transforms
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import pandas as pd
from sklearn.metrics import classification_report
```

— Config

```
FOLDER   = r"C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4"
EPOCHS   = 60
BATCH    = 16
LR       = 1e-4
NUM_CLASS = 3
LABEL_MAP = {'N': 0, 'M': 1, 'F': 2}
IDX_MAP  = {0: 'N', 1: 'M', 2: 'F'}
DEVICE   = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print(f"Using device: {DEVICE}")
if DEVICE.type == "cuda":
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"VRAM: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
```

—— Class weights (handles imbalance)

Inverse frequency from train distribution

```
WEIGHTS = {
    "TL": torch.tensor([37/22, 37/37, 37/11], dtype=torch.float32).to(DEVICE),  # N, M, F
    "TR": torch.tensor([40/21, 40/40, 40/9],  dtype=torch.float32).to(DEVICE),
    "BL": torch.tensor([1.0,  16.5,  5.0],   dtype=torch.float32).to(DEVICE),
    "BR": torch.tensor([1.0,  34.0,  5.0],   dtype=torch.float32).to(DEVICE),
}
```

—— Transforms

```
train_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.3),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    transforms.RandomRotation(degrees=8),
    transforms.RandomPerspective(distortion_scale=0.15, p=0.3),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

val_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

— Dataset

```
class DepthDataset(Dataset):
  def init(self, csv_path, img_dir, transform):
    self.df       = pd.read_csv(csv_path)
    self.img_dir  = img_dir
    self.transform = transform

def len(self):
    return len(self.df)

def getitem(self, idx):
    row  = self.df.iloc[idx]
    img  = Image.open(os.path.join(self.img_dir, row["filename"])).convert("RGB")
    img  = self.transform(img)
    labs = torch.tensor([LABEL_MAP[row["TL"]], LABEL_MAP[row["TR"]],
                         LABEL_MAP[row["BL"]], LABEL_MAP[row["BR"]]],
                        dtype=torch.long)
    return img, labs

train_ds = DepthDataset(os.path.join(FOLDER, "train.csv"), FOLDER, train_tf)
val_ds   = DepthDataset(os.path.join(FOLDER, "val.csv"),   FOLDER, val_tf)
test_ds  = DepthDataset(os.path.join(FOLDER, "test.csv"),  FOLDER, val_tf)

train_loader = DataLoader(train_ds, batch_size=BATCH, shuffle=True,
                          num_workers=2, pin_memory=True)
val_loader   = DataLoader(val_ds,   batch_size=BATCH, shuffle=False,
                          num_workers=2, pin_memory=True)
test_loader  = DataLoader(test_ds,  batch_size=BATCH, shuffle=False,
                          num_workers=2, pin_memory=True)
```

— Model

```
class DepthCNN(nn.Module):
  def init(self):
    super().init()
    backbone    = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    in_features = backbone.fc.in_features
    backbone.fc = nn.Identity()
    self.backbone = backbone
```

```
        # 4 independent classification heads (TL, TR, BL, BR)
        self.heads = nn.ModuleList([
            nn.Sequential(
                nn.Linear(in_features, 128),
                nn.ReLU(),
                nn.Dropout(0.3),
                nn.Linear(128, NUM_CLASS)
            ) for _ in range(4)
        ])

    def forward(self, x):
        feat = self.backbone(x)
        return [head(feat) for head in self.heads]

model = DepthCNN().to(DEVICE)

# ── Mixed precision scaler for RTX 4060 ──────────────────────

scaler    = torch.cuda.amp.GradScaler(enabled=(DEVICE.type == "cuda"))
optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)

regions = ["TL", "TR", "BL", "BR"]

def compute_loss(outputs, labels):
    total = 0
    for i, region in enumerate(regions):
        criterion = nn.CrossEntropyLoss(weight=WEIGHTS[region])
        total += criterion(outputs[i], labels[:, i])
    return total

def compute_acc(outputs, labels):
    correct, total = 0, 0
    for i in range(4):
        preds    = outputs[i].argmax(dim=1)
        correct += (preds == labels[:, i]).sum().item()
        total   += labels.size(0)
    return correct / total
```

— Training loop

```
best_val_acc = 0.0
best_path    = os.path.join(FOLDER, "best_model.pth")

print("\n" + "="*55)
print(f"{'Epoch':>6} | {'Train Loss':>10} | {'Train Acc':>9} | {'Val Acc':>7}")
print("="*55)

for epoch in range(1, EPOCHS + 1):
    # — Train
    model.train()
    t_loss, t_correct, t_total = 0, 0, 0
    for imgs, labels in train_loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        with torch.cuda.amp.autocast(enabled=(DEVICE.type == "cuda")):
            outputs = model(imgs)
            loss    = compute_loss(outputs, labels)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        t_loss   += loss.item()
        t_correct += sum((outputs[i].argmax(1) == labels[:, i]).sum().item() for i in range(4))
        t_total   += labels.size(0) * 4

    scheduler.step()
    train_acc  = t_correct / t_total
    train_loss = t_loss / len(train_loader)

    # — Validate
    model.eval()
    v_correct, v_total = 0, 0
    with torch.no_grad():
        for imgs, labels in val_loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            outputs  = model(imgs)
            v_correct += sum((outputs[i].argmax(1) == labels[:, i]).sum().item() for i in range(4))
            v_total   += labels.size(0) * 4
    val_acc = v_correct / v_total

    # — Save best model
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), best_path)
```

```
            flag = " ← best"
        else:
            flag = ""

    if epoch % 5 == 0 or epoch == 1:
        print(f"{epoch:>6} | {train_loss:>10.4f} | {train_acc:>9.4f} | {val_acc:>7.4f}{flag}")

print("="*55)
print(f"\nBest Val Accuracy: {best_val_acc:.4f}")
print(f"Model saved to: {best_path}")

# ── Test Evaluation ──────────────────────────────────────────

print("\n── Test Set Evaluation ──")
model.load_state_dict(torch.load(best_path))
model.eval()

all_preds  = [[] for _ in range(4)]
all_labels = [[] for _ in range(4)]

with torch.no_grad():
    for imgs, labels in test_loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        outputs = model(imgs)
        for i in range(4):
            all_preds[i].extend(outputs[i].argmax(1).cpu().tolist())
            all_labels[i].extend(labels[:, i].cpu().tolist())

for i, region in enumerate(regions):
    print(f"\n[{region}] Classification Report:")
    print(classification_report(all_labels[i], all_preds[i],
                                target_names=["Near", "Middle", "Far"],
                                zero_division=0))
```

make changes to it if you have any changes to make and give me the complete code

Download the fixed `train.py` above and replace the old one in your Task 4 folder. Here's a summary of every fix made:

What Changed and Why

Fix	Old (broken)	New (correct)
Windows multiprocessing	No `__main__` guard — spawned workers crashed	All training code inside `if __name__ == "__main__":`

Fix	Old (broken)	New (correct)
DataLoader workers	`num_workers=2` — causes crash on Windows	`num_workers=0` — single process, no spawning
pin_memory	Always `True` — warns when no GPU	`pin_memory=(DEVICE.type == "cuda")` — only when GPU present
GradScaler	Deprecated `torch.cuda.amp.GradScaler`	Fixed to `torch.amp.GradScaler("cuda", ...)`
autocast	Deprecated `torch.cuda.amp.autocast`	Fixed to `torch.amp.autocast("cuda", ...)`
Model loading	`torch.load(best_path)` — security warning	`torch.load(best_path, weights_only=True)`
WEIGHTS tensor	Created at module level before DEVICE confirmed	Moved inside `__main__` after DEVICE is set

Run It Now

```
python train.py
```

Expected output on your RTX 4050:

```
Using device: cuda
GPU: NVIDIA GeForce RTX 4050 Laptop GPU
VRAM: 6.0 GB

=======================================================
 Epoch | Train Loss | Train Acc |  Val Acc
=======================================================
     1 |     1.4231 |    0.5842 |  0.6250 <- best
     5 |     1.1023 |    0.6734 |  0.6750 <- best
    10 |     0.8654 |    0.7500 |  0.7000 <- best
```

Training should finish in **under 2 minutes** on your GPU. Paste the final test evaluation output and we'll move to the visualization overlay.

121 torchvision-0.20.1+cu121

(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python -c "import torch; print('cuda:', torch.cuda.is_available()); print('device:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU'); print('torch cuda version:', torch.version.cuda)"
cuda: True
device: NVIDIA GeForce RTX 4050 Laptop GPU
torch cuda version: 12.1
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4> python train.py

Using device: cuda
GPU: NVIDIA GeForce RTX 4050 Laptop GPU
VRAM: 6.4 GB

==========

==========

Epoch | Train Loss | Train Acc | Val Acc

```
  1 |     4.4413 |    0.3520 |  0.6500 <- best
  5 |     2.6786 |    0.8061 |  0.6500
 10 |     1.4146 |    0.9490 |  0.8000
 15 |     1.5489 |    0.9745 |  0.8500 <- best
 20 |     1.1304 |    0.9847 |  0.8500
 25 |     0.7555 |    0.9898 |  0.8500
 30 |     0.5560 |    0.9796 |  0.8500
 35 |     0.9284 |    0.9949 |  0.8500
 40 |     0.7376 |    0.9847 |  0.8250
 45 |     0.5362 |    0.9949 |  0.8250
 50 |     0.5742 |    1.0000 |  0.8500
 55 |     0.4738 |    1.0000 |  0.8500
 60 |     0.5194 |    0.9949 |  0.8500
```

==========

Best Val Accuracy: 0.8500
Model saved: C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\best_model.pth

── Test Set Evaluation ──

[TL] Classification Report:
precision recall f1-score support

```
     Near       0.50      0.50      0.50         2
   Middle       0.83      0.56      0.67         9
      Far       0.00      0.00      0.00         0

 accuracy                           0.55        11
```

macro avg 0.44 0.35 0.39 11
weighted avg 0.77 0.55 0.64 11

[TR] Classification Report:
precision recall f1-score support

```
     Near       0.33      1.00      0.50         1
   Middle       1.00      0.67      0.80         9
      Far       0.00      0.00      0.00         1
```

```
accuracy                           0.64        11
```

macro avg 0.44 0.56 0.43 11
weighted avg 0.85 0.64 0.70 11

[BL] Classification Report:
Traceback (most recent call last):
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\train.py", line 192, in <module>
print(classification_report(all_labels[i], all_preds[i],
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\sklearn\utils_param_validation.py", line 218, in wrapper
return func(*args, **kwargs)
File "C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4\venv\lib\site-packages\sklearn\metrics_classification.py", line 2970, in classification_report
raise ValueError(
ValueError: Number of classes, 2, does not match size of target_names, 3. Try specifying the labels parameter
(venv) PS C:\Users\gbhan\COLLEGE NOTES ARRANGING\SECOND SEMESTER\Image-Processing and Computer Vision\TASKS\Task 4>

Good — training **worked**. The crash is only in the reporting section, not in the model training itself.

What your result means

Your model trained successfully and reached **0.85 validation accuracy**, but training accuracy went to nearly **1.00**, which is a classic sign of overfitting on a tiny dataset. Small datasets with a ResNet18 backbone can do this very easily even with augmentation and dropout.[99] [100]

The final crash happened because `classification_report` expects the number of labels to match `target_names`, and your BL/BR test split does not contain all three classes. Scikit-learn allows fixing this by explicitly passing `labels=[0,1,2]`.[101] [102]

Replace only the reporting block

You do **not** need to retrain if `best_model.pth` already exists. Replace the final reporting part of `train.py` with this:

```
# ── Test Evaluation ──────────────────────────────────────────────────────
print("\n── Test Set Evaluation ──")
model.load_state_dict(torch.load(best_path, weights_only=True))
model.eval()

all_preds  = [[] for _ in range(4)]
all_labels = [[] for _ in range(4)]
```

```
with torch.no_grad():
    for imgs, labels in test_loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        outputs = model(imgs)
        for i in range(4):
            all_preds[i].extend(outputs[i].argmax(1).cpu().tolist())
            all_labels[i].extend(labels[:, i].cpu().tolist())

for i, region in enumerate(regions):
    print(f"\n[{region}] Classification Report:")
    print(classification_report(
        all_labels[i],
        all_preds[i],
        labels=[0, 1, 2],
        target_names=["Near", "Middle", "Far"],
        zero_division=0
    ))
```

That is the exact fix recommended by scikit-learn's `classification_report` behavior for missing classes in `y_true` or `y_pred`.[102] [101]

Better move: create a test-only script

Don't retrain again just to print metrics. That would be dumb. Your best model is already saved. Create a new file called `evaluate.py` with the same dataset/model definitions as `train.py`, load `best_model.pth`, and run only test evaluation. This avoids wasting GPU time and keeps your workflow cleaner. Since `best_model.pth` was already saved at epoch 15, you can evaluate directly from that file.[99]

Brutal assessment

Your validation score looks decent, but don't fool yourself — with 70 images and heavily imbalanced bottom quadrants, this is not a strong model. It is a **coursework prototype**, not a robust estimator. The right academic move now is:

- finish evaluation,
- generate a few example predictions,
- show overlay visualization,
- explain dataset limitations honestly.[103]

Next step

Do one of these:

1. Patch `train.py` with the block above and run it again, or
2. Ask me and I'll give you a clean `evaluate.py` plus an **overlay visualization script** so you can produce report-ready example outputs.

⁂

1. PM05-Monocular-Depth-Estimation.pdf
2. PM05-Monocular-Depth-Estimation.pdf
3. https://www.youtube.com/watch?v=mTVf7BN7S8w
4. https://www.datacamp.com/tutorial/complete-guide-data-augmentation
5. https://www3.tuhh.de/sts/hoou/data-quality-explored/2-3-2-augmentation.html
6. https://arxiv.org/html/2502.18691v1
7. https://datature.com/blog/image-augmentation-for-machine-learning-techniques-examples-code
8. https://www.iese.fraunhofer.de/blog/data-augmentation-with-generative-ai/
9. https://discuss.pytorch.org/t/increase-number-of-images/132075
10. https://www.facebook.com/groups/ComputerVisionGroup/posts/2413525112125009/
11. https://encord.com/blog/data-augmentation-guide/
12. https://d2l.ai/chapter_computer-vision/image-augmentation.html
13. PM05-Monocular-Depth-Estimation.pdf
14. IMG_20260526_151018.jpg
15. https://www3.tuhh.de/sts/hoou/data-quality-explored/2-3-2-augmentation.html
16. https://www.youtube.com/watch?v=H2IFv_GBdUQ
17. https://nlp.johnsnowlabs.com/docs/en/alab/tasks
18. https://www.facebook.com/Todoist/videos/tutorial-todoist-sorting-options/214647413409020/
19. https://techcommunity.microsoft.com/t5/planner/once-more-edit-labels/m-p/3961249
20. https://www.instagram.com/reel/DXcLFY8iCNt/
21. https://github.com/HumanSignal/label-studio/issues/2345
22. https://www.servicenow.com/community/developer-forum/couldn-t-add-or-edit-labels-in-visual-task-boards/m-p/1414208
23. https://apps.apple.com/us/app/todoist-to-do-list-calendar/id572688855
24. https://forum.asana.com/t/api-query-original-task-name/470992
25. https://www.youtube.com/watch?v=_4VdQYFpaKo
26. https://docs.python.org/3/tutorial/venv.html
27. https://www.w3schools.com/python/python_virtualenv.asp
28. https://realpython.com/python-virtual-environments-a-primer/
29. https://docs.python.org/3/library/venv.html
30. https://docs.python.org/uk/3.14/library/venv.html
31. https://docs.python.org/uk/3.10/library/venv.html
32. https://docs.python.org/3.9/library/venv.html
33. https://docs.python.org/es/3/library/venv.html
34. https://docs.python.org/uk/3/library/venv.html
35. https://stackoverflow.com/questions/43069780/how-to-create-virtual-env-with-python-3

36. PM05-Monocular-Depth-Estimation.pdf
37. https://superb-ai.com/en/resources/blog/best-practices-for-annotating-images-in-computer-vision-datasets
38. https://dagshub.com/blog/best-practices-for-managing-computer-vision-projects/
39. https://www.ce.cit.tum.de/fileadmin/w00cgn/air/Personal_Files/WalterZimmer/final_masters_thesis_ahmed_ghita_compressed_default.pdf
40. https://arxiv.org/html/2507.22361v1
41. https://www.sciencedirect.com/science/article/pii/S0925231223006811
42. https://nightjar.so/blog/ai-camera-angle-control-guide
43. https://www.picsellia.com/post/public-datasets-computer-vision
44. https://aimagelab-legacy.ing.unimore.it/imagelab/pubblicazioni/iciap05_cr.pdf
45. https://arxiv.org/html/2507.02393v1
46. https://labelyourdata.com/articles/3d-object-detection
47. https://github.com/ultralytics/ultralytics/issues/6563
48. https://dt-labs.ai/blog/multi-camera-multi-object-tracking/
49. https://labelstud.io/blog/getting-started-with-object-detection/
50. https://www.reddit.com/r/computervision/comments/1k3itiw/imagedatasetcreation_best_practices/
51. https://www.youtube.com/watch?v=v0ssiOY6cfg
52. PM05-Monocular-Depth-Estimation.pdf
53. https://gist.github.com/Zakaria-pro/63eedd15537769c22e6996630da65564
54. https://portrait.gitee.com/Gramyd/classifying-image/blob/master/check_images_solution.py
55. https://github.com/davidbroetje/udacity-ai-programming-with-python/blob/master/intropylab-classifying-images/check_images_hints.py
56. https://gist.github.com/Aimuhire/443a7db77ebc0dbfbd79382aeb7576f9
57. https://docs.python.org/3/library/venv.html
58. https://pastebin.com/JLWg3390
59. https://github.com/udacity/AIPND-revision/blob/master/intropyproject-classify-pet-images/check_images.py
60. https://portrait.gitee.com/Gramyd/classifying-image/blob/master/print_functions_for_lab_checks.py
61. https://programminghistorian.org/en/lessons/computer-vision-deep-learning-pt2
62. https://pastebin.com/418sYiqz
63. PM05-Monocular-Depth-Estimation.pdf
64. https://www.reddit.com/r/math/comments/13rj9wa/saw_this_graphic_showing_supposedly_the_uk_being/
65. https://community.adobe.com/questions-712/bug-splits-my-image-into-four-quadrants-1117268
66. https://denovosoftware.com/faq/quadrants-earlier-versions-options/

67. https://www.facebook.com/chipconleyauthor/posts/my-love-affair-with-4-quadrantstheres-something-wildly-comforting-about-a-22-gra/1117183230227008/
68. https://www.mathworks.com/matlabcentral/answers/426804-splitting-image-into-4-quadrants-up-down-right-and-left
69. https://www.ibm.com/think/insights/how-to-make-decisions-with-4-quadrants
70. https://www.bishophouse.com/wp-content/uploads/2013/02/Effective-Personal-Management-with-Covey-The-4-Quads.pdf
71. https://www.youtube.com/watch?v=xthNZqWCN2I
72. https://github.com/davidbroetje/udacity-ai-programming-with-python/blob/master/intropylab-classifying-images/check_images_hints.py
73. https://programminghistorian.org/en/lessons/computer-vision-deep-learning-pt2
74. https://github.com/udacity/AIPND-revision/blob/master/intropyproject-classify-pet-images/check_images.py
75. https://docs.python.org/3/library/venv.html
76. https://pastebin.com/JLWg3390
77. https://gist.github.com/Zakaria-pro/63eedd15537769c22e6996630da65564
78. https://realpython.com/python-virtual-environments-a-primer/
79. https://gist.github.com/Aimuhire/443a7db77ebc0dbfbd79382aeb7576f9
80. https://portrait.gitee.com/Gramyd/classifying-image/blob/master/check_images_solution.py
81. https://portrait.gitee.com/Gramyd/classifying-image/blob/master/print_functions_for_lab_checks.py
82. PM05-Monocular-Depth-Estimation.pdf
83. https://discuss.pytorch.org/t/install-pytorch-with-cuda-12-1/174294
84. https://pytorch.org/get-started/locally/
85. https://github.com/pytorch/pytorch/issues/13850
86. https://discuss.pytorch.org/t/errors-when-using-num-workers-0-in-dataloader/97564
87. PM05-Monocular-Depth-Estimation.pdf
88. https://stackoverflow.com/questions/77068908/how-to-install-pytorch-with-cuda-support-on-windows-11-cuda-12-no-matching
89. https://forums.fast.ai/t/dataloader-on-windows-with-num-workers-0/88734
90. https://www.facebook.com/groups/2059467967664033/posts/4038789533065190/
91. https://mmcv.readthedocs.io/en/master/_modules/torch/utils/data/dataloader.html
92. https://pytorch.org/get-started/previous-versions/
93. https://www.youtube.com/watch?v=kWVgvsejXsE
94. https://deepnote.com/blog/ultimate-guide-to-pytorch-library-in-python
95. https://discuss.pytorch.org/t/torch-dataloader-num-workers-0-not-spawning-workers/106114
96. https://www.reddit.com/r/pytorch/comments/k6o3yh/cant_use_num_workers_0_with_cuda/

97. https://stackoverflow.com/questions/53998282/how-does-the-number-of-workers-parameter-in-pytorch-dataloader-actually-work
98. https://www.linkedin.com/pulse/pytorch-dataloader-usage-windows-xiao-liu-1
99. https://codesignal.com/learn/courses/improving-neural-networks-with-pytorch/lessons/early-stopping-in-pytorch-preventing-overfitting-during-training
100. https://discuss.pytorch.org/t/severe-overfitting-with-resnet18-on-small-dataset-despite-regularization/224004
101. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html
102. https://github.com/scikit-learn/scikit-learn/issues/16984
103. PM05-Monocular-Depth-Estimation.pdf
104. https://github.com/scikit-learn/scikit-learn/issues/7751
105. https://stackoverflow.com/questions/48493322/classification-report-labels-and-target-names/60712507
106. https://www.reddit.com/r/learnmachinelearning/comments/1l4bfzl/is_my_neural_net_pytorch_model_overfitting/
107. https://scikit-learn.org/0.16/modules/generated/sklearn.metrics.classification_report.html
108. https://github.com/huggingface/pytorch-openai-transformer-lm/issues/31
109. https://github.com/scikit-learn/scikit-learn/issues/3123
110. https://github.com/scikit-learn/scikit-learn/issues/13055
111. https://hyperskill.org/learn/step/29339
112. https://github.com/scikit-learn/scikit-learn/issues/9842
113. https://stackoverflow.com/questions/57916221/number-of-classes-4-does-not-match-size-of-target-names-6-try-specifying-the
114. https://stackoverflow.com/questions/61400835/is-it-possible-to-be-overfitted-with-low-training-accuracy